Stockbot
수집·SQL·운영 조회와 GitHub Actions CI/CD

원본 저장소와 개발 이력을 보고 싶으시면, 이력서의 연락처로 GitHub 초대에 사용할 이메일 주소를 알려주세요. 확인 후 초대하겠습니다.

국내 주가·거래량을 수집해 여러 해의 흐름과 기간별 통계를 같은 기준으로 비교하는 개인 리서치 도구입니다. **HTTP·파일 수집 → Rust Job/Step → PostgreSQL 시계열·집계 → Actix API → Yew/Rust WASM 조회·운영 화면**을 연결했습니다.

공개 검토용 소스 사본은 원본 main 18c1f8e의 처리 코드를 복사하고 운영 식별값·주소 등을 예시로 바꾼 저장소입니다. 원본 저장소·개발 이력은 비공개이며 위 안내에 따라 검토 초대를 요청할 수 있습니다. 사본은 원본 PR·Actions 실행 이력을 가져오지 않았고 자동 실행은 비활성 상태입니다.

1. 실행 단계·API·웹을 공통 계약으로 연결

Spring Batch를 참고해 Batch·Job 실행 인터페이스와 Step read/process/write를 나눴습니다. 실행부·공유 모델·클라이언트·유틸리티를 9개 Rust crate로 구성하고 Cargo feature로 DB·브라우저 의존성을 분리했습니다.

배치 종류·국가·실행 상태는 BatchParams의 JSON·utoipa·CLI 계약을 공유합니다. OpenAPI·Swagger UI 등록과 Yew/WASM 화면도 연결했습니다. 원래 Rust 구현은 2022-2025년, 2026년 AI 에이전트 보조 Axum·React 전환은 별도 브랜치입니다.

2. SQL로 원천 시세를 비교 가능한 조회 모델로

시세의 종목·날짜 복합키와 RANGE 파티션, 방향 구간 bound의 종목·가격종류·시작일 키, 기간별 통계의 복합키로 원천·파생·집계를 나눴습니다. 복사본의 SQL·스키마 문서에는 DDL 기반 관계도와 조회 흐름을 함께 제공합니다.

저장된 시세에서 유도한 거래소별 거래일 RANK로 구간 길이를 거래일 축에서 다룹니다. 종목별 LAG와 0·초기값 분기로 거래량 변화율을 계산합니다. bound는 같은 방향이 이어지는 구간이며 두 끝점의 y=mx+b는 비교용 특성입니다.

거래량은 월·주·분기 UNION ALL·RANK → CTE·sum(COUNT(*)) OVER로 여러 연도에서 기준별 1위였던 기간의 반복 빈도·비중 → 공동 1위 기간을 array_agg로 반환하는 단계로 나눴습니다. bound 변화율·길이·기간 축은 공통 PL/pgSQL 집계 함수로 재사용했습니다. Rust 배치 CALL과 API view 조회에 실제 연결된 SQL입니다.

3. 수집·웹 스크래핑·비동기 작업의 경계

기본 가격 경로는 Naver HTTP 요청 → JSON 계열 응답의 날짜·가격·거래량 파싱입니다. KRX는 OTP 발급·Excel 다운로드와 Sheet1 종목 상태 파싱을 나눴습니다. 종목별 최종 저장일과 upsert로 재수집을 다뤘습니다.

Tokio 작업과 async channel::bounded(tx/rx)로 생산·소비를 나누고 여러 소비자가 계산·DB 저장을 맡습니다. bound의 용량 1 채널은 send().await로 생산 측을 기다리게 합니다. 가격 요청 자체의 종목별 순차 await와 별도 송신 작업은 남아 있습니다.

HTML 뉴스 파서의 날짜·매체·제목·링크 선택자도 별도 구현했습니다. 이 모듈은 기본 main 미등록·소비자 await 누락이 있어 현재 운영 수집 완료 기능으로 포함하지 않습니다.

4. 배치 실행이력 저장 → API → WASM 운영 조회

system.batch·batch_step의 FK와 실행계획·heartbeat DDL로 실행 부모·단계·계획·상태를 나눴습니다. Job/Step 실행 시각·소요시간 기록 → 이력 JOIN·정렬·페이지 DAO → Actix GET /system/logs/batch → Rust/WASM 운영 표로 연결합니다.

START/EXIT 실행계획과 스케줄러 heartbeat 시각도 조회합니다. 별도 중앙 로그 수집 서버 대신 직접 만든 DB 실행이력·운영 조회 구조입니다. tracing·Actix Logger는 프로세스 로그를 맡으며 원본 오류 경로의 완료 기록·재시작을 모두 보장하지는 않습니다.

5. GitHub Actions CI/CD와 공통 ARM64 빌드

공통 ARM64 툴체인·linker·sccache → 서비스별 선행 빌드층 → 실행 이미지 분리로 구성했습니다. CI/CD 문서에는 실제 네 workflow의 차이를 정리했습니다.

API: ARM64 → ECR → S3 → CodeDeploy, 스케줄러 이미지·배포, 배치 이미지 ECR push, Rust/WASM dist 추출·S3 배포를 분리했습니다. 백엔드 세 경로는 latest·commit 태그, 웹은 latest 로컬 이미지의 정적 산출물을 사용합니다. 원본에는 네 경로 모두 과거 성공 실행 기록이 있습니다.

조건·계획 ID를 user-data로 전달하고 기존 배치 EC2 시작 → 부팅 후 실행 → 완료 API → EC2 정지로 연결했습니다. 과거 구성은 DB / API+스케줄러 / 배치의 EC2 3역할과 S3 웹·캐시·배포 묶음 3역할입니다. 공개 사본은 실행 설정을 읽는 자료이며 AWS 자원을 새로 배포하지 않았습니다.

6. 원본과 공개 사본·남은 범위

원본/사본 파일 해시와 변환 기록을 제공합니다. 소스 처리 알고리즘을 다시 쓴 독립 예제가 아니라 원본 main의 검토용 복사본입니다. 원본 개발 이력·별도 Go/Spring 실험·2026년 후속 브랜치는 초대 후 확인할 수 있습니다.

UNNEST·CSV/COPY는 별도 적재 구현·실험입니다. 파티션 연말 경계, LAG 증분 첫 행, 기존 bound 꼬리 재계산, 최근 스냅샷 OHLC 변환에는 보완 과제가 있습니다. 투자 수익·대규모 트래픽 성과로 설명하지 않습니다.